

Java Lab Record: Arrays / Collections - Valid Parentheses

Objective / Aim

To design and implement a Java program that checks whether a string of brackets (,), { , }, [and] is **valid**, using a **stack** built with the Collection Framework class `ArrayDeque` (through the `Deque` interface).

Problem Statement

Given a string containing just the characters (,), { , }, [and] , determine if the input string is valid. An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.

Note that an empty string is also considered valid.

Expected Output

Input: () Output: valid
Input: ({) } Output: Not valid

System Requirements

Hardware

Standard PC / Laptop
Intel Core i3 / i5 processor or equivalent
Minimum 4 GB RAM
Minimum 500 MB free storage

Software

Operating System : Windows / Linux / macOS
Java Development Kit : JDK 17 or JDK 21
Text Editor or IDE : Eclipse, IntelliJ IDEA, NetBeans, or VS Code
Command Prompt / Terminal

Algorithm / Logic Description

1. Create a class named ValidParentheses.
2. Create a method isValid(input) that returns true or false.
3. Create an empty stack (Deque implemented by ArrayDeque).
4. Read the input one character at a time:
 - a. If it is a space, skip it.
 - b. If it is an opening bracket ({ [-> push it onto the stack.
 - c. If it is a closing bracket) }] ->
 - if the stack is empty, return false.
 - pop the top bracket from the stack.
 - if it is not the matching opening bracket, return false.
 - d. Any other character is ignored.
5. After the loop, if the stack is empty, return true (valid). Otherwise return false (some bracket was never closed).
6. In main(), read a string using Scanner, call isValid(), and print "valid" or "Not valid".

Real-time Application

Syntax checking in compilers and code editors (matching brackets), validating mathematical expressions, checking HTML/XML tags, and undo/redo style features that use a stack.

ValidParentheses.java

Source Code

```
package collectionspractice;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Scanner;

public class ValidParentheses {

    public static boolean isValid(String input) {
        // ArrayDeque works as a stack: the last bracket opened must close
        // first (LIFO).
        // A queue checks brackets in arrival order, so it cannot validate
        // nested brackets.
        Deque<Character> openingBrackets = new ArrayDeque<>();

        for (char bracket : input.toCharArray()) {
            // Ignore spaces so inputs such as "( )" are accepted as shown in
            // the sample.
            if (Character.isWhitespace(bracket)) {
                continue;
            }

            if (bracket == '(' || bracket == '{' || bracket == '[') {
```

```

        openingBrackets.push(bracket);
    } else if (bracket == '(' || bracket == '[' || bracket == '{') {
        if (openingBrackets.isEmpty()) {
            return false;
        }

        char openingBracket = openingBrackets.pop();
        if ((bracket == ')') && openingBracket != '(')
            || (bracket == '}' && openingBracket != '{')
            || (bracket == ']' && openingBracket != '[') {
            return false;
        }
    }
    // Other characters are ordinary text, so they do not affect
    bracket matching.
}

// Any bracket left in the stack was opened but never closed.
return openingBrackets.isEmpty();
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Enter brackets: ");
    String input = scanner.nextLine();

    if (isValid(input)) {
        System.out.println("valid");
    } else {
        System.out.println("Not valid");
    }

    scanner.close();
}
}

```

1. Concept and Exception in Simple Words

Collection Framework concepts used

Collection -> the root interface for groups of objects in Java.

Deque -> "double-ended queue": you can add and remove from both ends. When we use push() and pop() on one end, it behaves like a STACK.

ArrayDeque -> a Collection class that implements Deque. It is fast and is the recommended class to use as a stack in Java.

Stack (LIFO)-> Last In, First Out. The bracket opened LAST must be closed FIRST, exactly what nested brackets need.

Related Collection classes in your syllabus

ArrayList -> resizable array; fast access by index.

LinkedList -> nodes linked together; it can also work as a Deque/Queue, so it could replace ArrayDeque in this program.

HashSet -> stores unique items with no order.
 TreeSet -> stores unique items in sorted order.
 PriorityQueue -> a queue where the smallest (or highest priority) item
 comes out first.

This program uses `ArrayDeque` because a stack is the right tool. A normal queue (FIFO) cannot be used, because it gives brackets back in arrival order, not in the reverse order that nested brackets need.

Why a stack works

Input: ({ }) '(' push -> [(] '{' push -> [({] '}' pop '{' matches -> [(] ')' pop '(' matches -> [] Stack empty -> valid	Input: ({) } '(' push -> [(] '{' push -> [({] ')' pop '{' ... does NOT match -> Not valid
--	--

Exception in simple words

An exception is an unexpected problem that stops the normal flow of a program.

Exception	Kind	Where it comes from
<code>NoSuchElementException</code>	Unchecked	<code>pop()</code> throws it if the stack is empty. This program avoids it by checking <code>isEmpty()</code> before every <code>pop()</code> Would happen if input were null and
<code>NullPointerException</code>	Unchecked	<code>toCharArray()</code> was called. <code>Scanner.nextLine()</code> never gives null, so it is safe here
<code>NoSuchElementException</code> (Scanner)	Unchecked	<code>nextLine()</code> throws it if no input is available at all (for example, input stream closed)

No try-catch is needed for normal use. The empty-stack check in the program is the "safety guard" that prevents the exception.

2. Step-wise Execution (Input "({) }")

1. `main()` starts and prints "Enter brackets: ".
2. Scanner reads the line: `( { ) }`
3. `isValid()` is called. An empty stack is created: `[]`
4. Character `'('` -> opening bracket -> push. Stack: `[ ( ]`
5. Character `' '` -> space -> skipped.
6. Character `'{'` -> opening bracket -> push. Stack: `[ ( { ]`
7. Character `' '` -> space -> skipped.
8. Character `')'` -> closing bracket.
 Stack is not empty -> pop gives `'{'`.
 Closing is `')` but popped is `'{'` (needs `'('`) -> mismatch.
 `isValid()` returns false immediately.
9. `main()` prints "Not valid".
10. Scanner is closed and the program ends.

Step-wise execution for Input "()"

- ```

1. '(' -> push. Stack: [(]
2. ' ' -> skipped.
3. ')' -> pop gives '(' -> matches. Stack: []
4. End of input. Stack is empty -> return true.
5. main() prints "valid".

```

### 3. Algorithm

START

- ```
1. READ input string
2. CALL isValid(input)
3. IF result is true: PRINT "valid"
   ELSE: PRINT "Not valid"
```

STOP

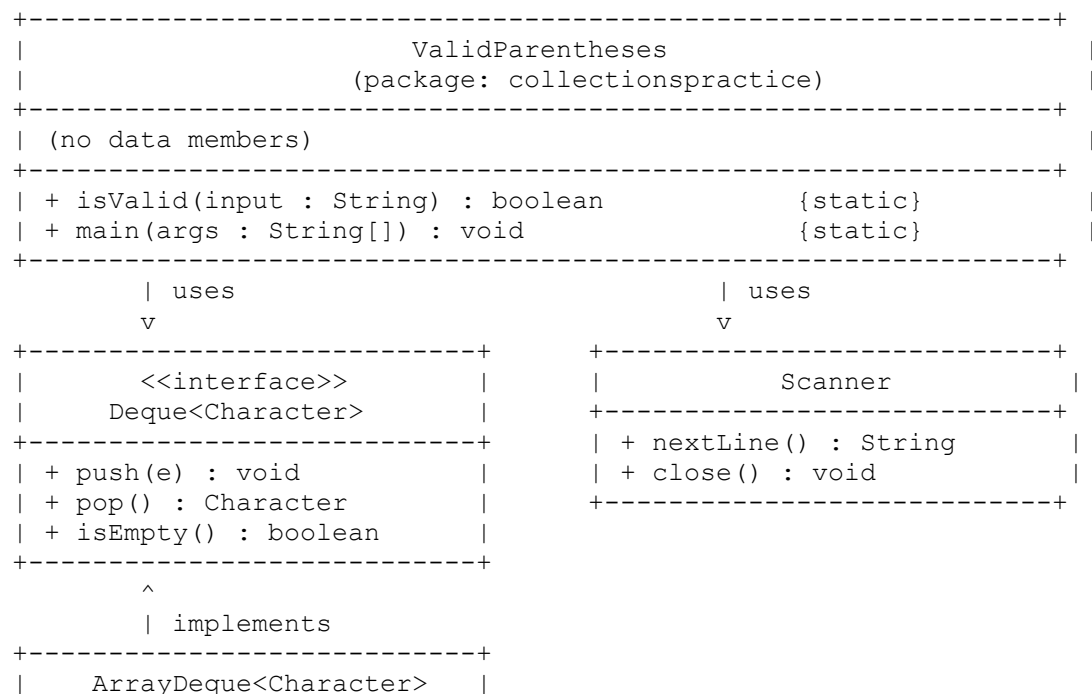
```
isValid(input)
```

- ```

1. stack = new empty Deque (ArrayDeque)
2. FOR each character ch in input:
 IF ch is a space: CONTINUE
 IF ch is '(' or '{' or '[':
 PUSH ch onto stack
 ELSE IF ch is ')' or '}' or ']':
 IF stack is empty: RETURN false
 open = POP from stack
 IF ch and open are not a matching pair: RETURN false
3. RETURN (stack is empty)

```

## 4. UML Class Diagram



+-----+

**Legend:** + public, {static} belongs to the class itself.

---

---

## Compilation and Execution

Folder structure:

```
project/
|-- collectionspractice/
 |-- ValidParentheses.java
```

Compile:

```
javac -d out collectionspractice/ValidParentheses.java
```

Run:

```
java -cp out collectionspractice.ValidParentheses
```

## Sample Output

Run 1:

```
Enter brackets: ()
valid
```

Run 2:

```
Enter brackets: ({) }
Not valid
```

Run 3:

```
Enter brackets: {[()]}
Valid
```

## Output:

```
PS C:\Users\Admin\OneDrive\Documents\basicjava> c;; cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.ValidParentheses'
Enter brackets: {
Not valid
PS C:\Users\Admin\OneDrive\Documents\basicjava> c;; cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.ValidParentheses'
Enter brackets: {}
valid
PS C:\Users\Admin\OneDrive\Documents\basicjava> c;; cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.ValidParentheses'
Enter brackets: {234,4535}
Not valid
PS C:\Users\Admin\OneDrive\Documents\basicjava> c;; cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.ValidParentheses'
Enter brackets: [23,425]
valid
PS C:\Users\Admin\OneDrive\Documents\basicjava> c;; cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'collectionspractice.ValidParentheses'
Enter brackets: {e34,45346,52,54..}
valid
PS C:\Users\Admin\OneDrive\Documents\basicjava>
```

## ✦ Result

The Valid Parentheses program was successfully implemented in Java using the Collection Framework. An `ArrayDeque` (through the `Deque` interface) was used as a stack to match opening and closing brackets in the correct order. The program printed "valid" for the input ( ) and "Not valid" for the input ( { } ), matching the expected output.